NoteNest.git

SYSTEM PURPOSE: NoteNest is a web application designed to store and manage notes. It allows users to create, edit, and delete notes, as well as upload files. The system is intended for personal use, providing a simple and intuitive interface for users to organize their notes and files.

TECH STACK: - backend: Python - frontend: JavaScript, CSS, HTML - database: Not implemented in code - libraries: Not implemented

MODULES: Application Module: The Application Module is responsible for handling user requests, processing data, and interacting with the database. It contains the main application logic and is the core of the system. Files: app.py

Template Module: The Template Module is responsible for rendering HTML templates and providing a user interface for the application. It contains the templates for the application's pages and is used to display data to the user. Files: templates\admin.html, templates\chatbot.html, templates\index.html, templates\login.html, templates\notes.html, templates\profile.html, templates\register.html, templates\upload.html

API ROUTES: - GET /: The root endpoint of the application, which renders the index.html template and displays the application's main page. - GET /login: The login endpoint, which renders the login.html template and provides a form for users to enter their login credentials. - GET /register: The registration endpoint, which renders the register.html template and provides a form for users to create a new account.

DATA FLOW: - When a user requests a page, the Application Module processes the request and retrieves the necessary data from the database (if implemented). The Template Module then renders the corresponding HTML template and displays the data to the user. - When a user submits a form, the Application Module processes the form data and updates the database (if implemented). The Template Module then renders the updated HTML template and displays the changes to the user.

ARCHITECTURE: The NoteNest system follows a simple client-server architecture, where the client is the web browser and the server is the Python application running on the server-side. The system uses a request-response model, where the client sends a request to the server and the server responds with the requested data.

SECURITY: - Not implemented in code

IMPROVEMENTS: - Implement a database to store user data and notes. - Add authentication and authorization mechanisms to secure the application. - Improve the user interface and add more features to enhance the user experience.